

PYTHON DATA STRUCTURE

Data Structures are a way of organising data so that it can be accessed more efficiently depending upon the situation. Data Structures are fundamentals of any programming language around which a program is built. Python helps to learn the fundamentals of these data structures in a simpler way as compared to other programming languages.

1. String
2. List
3. Tuple
4. Dictionary
5. set

1. Strings in Python:

A string is a sequence of characters that can be a combination of letters, numbers, and special characters. It can be declared in python by using single quotes, double quotes, or even triple quotes. These quotes are not a part of a string, they define only the starting and end of the string. Strings are immutable, i.e., they cannot be changed. Each element of the string can be accessed using indexing or slicing operations.

```
In [13]: x="This is a string";x
Out[13]: 'This is a string'
```

String operations

a. String comparison

```
In [6]: s1="a"
        s2="A"
        s1==s2
```

```
Out[6]: False
```

```
In [7]: s1>s2
```

```
Out[7]: True
```

b. Split method

```
In [8]: s="To be or not to be"
s.split()
```

```
Out[8]: ['To', 'be', 'or', 'not', 'to', 'be']
```

```
x=" python \ programming "
```

```
x.split(sep=" \ ")
```

```
[' python ', 'programming ']
```

c. “+” operator- concatenate

```
In [9]: g1="good"
g2="morning"
g1+g2
```

```
Out[9]: 'goodmorning'
```

d. * operator

```
In [10]: s="hello"
s*3
```

```
Out[10]: 'hellohellohello'
```

e. In operator

```
In [11]: s="python programming"
"python" in s
```

```
Out[11]: True
```

- f. **find()** : It returns the lowest index where the substring starts in the string or returns -1 if it's not found in the string.

```
] : k="swathi"
```

```
] : k.find("p")
```

```
] : -1
```

- g. **rfind()** : It gives the highest index where the substring starts in the string or returns -1 if it's not found in the string.

```
In [24]: st="python programming problem"
st.rfind("pro")
```

```
Out[24]: 19
```

- h. **count()** : returns the number of occurrences of the given substring.

```
In [25]: st="python programming problem"
st.count("pro")
```

```
Out[25]: 2
```

- i. **capitalize ()**: To make the most first letter of a sentence capital.

```
In [27]: st="hello"
st.capitalize()
```

```
Out[27]: 'Hello'
```

- j. **lower()** : To convert all into lower case.

```
In [28]: st="HELLO"
st.lower()
```

```
Out[28]: 'hello'
```

- k. **upper()**: converts all letters into capital.

```
In [29]: st="hello"
st.upper()
```

```
Out[29]: 'HELLO'
```

- l. **title()** : capitalize the first letter of each word.

```
In [30]: st="python programming problem"
st.title()
```

```
Out[30]: 'Python Programming Problem'
```

- m. **swapcase()** : Converts lower into upper and vise versa.

```
In [31]: st="InDIa"
st.swapcase()
```

```
Out[31]: 'iNdIA'
```

2. Lists

A list is a **data structure in Python that is a mutable, or changeable, ordered sequence of elements**. Each element or value that is inside of a list is called an item. Just as strings are defined as characters between quotes, lists are defined by having values between square brackets []

List indices[]

Python starts with counting from 0. So, asking for first element 0 gives the first element. And if the indices from -ve , they indicate positions counting from the end of the list.

```
In [34]: x=["first","second","third"]
         x[1]
Out[34]: 'second'

In [35]: x[0]
Out[35]: 'first'

In [36]: x[-1]
Out[36]: 'third'
```

Slicing of list

Slicing is a flexible tool to build new lists out of an existing list. **List[index1:index2]** to extract all the items **including index1 upto index2**.

```
In [39]: X=[1,2,3,4,5,6,7,8,9,10]
         X[2:6]
Out[39]: [3, 4, 5, 6]
```

Indices can't be negative order

```
In [44]: X[6:2]
Out[44]: []
```

We can use -ve indices

```
In [43]: X[1:-1]
Out[43]: [2, 3, 4, 5, 6, 7, 8, 9]
```

Slicing a list with single indices is possible. Leaving out index 2 to go to the end of the list.

```
In [45]: X[:6]
Out[45]: [1, 2, 3, 4, 5, 6]

In [46]: X[2:]
Out[46]: [3, 4, 5, 6, 7, 8, 9, 10]
```

Modifying a list

- a. Using **list[index]** we can modify a list by pulling the index on the left side of the assignment operator.

```
In [48]: X=[1,2,3]
         X[0]="TWO"
         X
```

```
Out[48]: ['TWO', 2, 3]
```

b. To *append* list to the **end**

```
In [51]: X=[1,2,3]
         X[len(X):]=[4,5,6]
         X
```

```
Out[51]: [1, 2, 3, 4, 5, 6]
```

c. To *append* list to the **front**

```
In [52]: X=[1,2,3]
         X[:0]=[4,5,6]
         X
```

```
Out[52]: [4, 5, 6, 1, 2, 3]
```

d. to **remove** element from list

```
In [57]: X=[1,2,3,4,5]
         X.remove(3)
         X
```

```
Out[57]: [1, 2, 4, 5]
```

e. remove elements from the list

```
In [58]: X=[1,2,3,4,5]
         X[1:-1]=[]
         X
```

```
Out[58]: [1, 5]
```

f. **del** statement

```
In [59]: X=[1,2,3,4,5]
         del X[1]
         X
```

```
Out[59]: [1, 3, 4, 5]
```

g. **append()**

```
In [61]: X=[1,2,3]
         Y=[4,5,6]
         X.append(Y)
         X
```

```
Out[61]: [1, 2, 3, [4, 5, 6]]
```

h. extend()

```
In [63]: X=[1,2,3]
         Y=[4,5,6]
         X.extend(Y)
         X
```

```
Out[63]: [1, 2, 3, 4, 5, 6]
```

i. insert()

```
In [66]: X=[1,2,3]
         X.insert(2,"Hello")
         X
```

```
Out[66]: [1, 2, 'Hello', 3]
```

j. reverse()

```
In [68]: X=[1,8,5,0,2]
         X.reverse()
         X
```

```
Out[68]: [2, 0, 5, 8, 1]
```

k. sort()

```
In [71]: X=[1,6,2,9,7,5,8,3]
         X.sort()
         X
```

```
Out[71]: [1, 2, 3, 5, 6, 7, 8, 9]
```

l. To find the index

```
In [72]: X=[1,2,3,4,5,6,7,8,9,10]
         X.index(1)
```

```
Out[72]: 0
```

m. To count the occurrence of a value

```
In [74]: X=[1,3,6,2,3,9,7,3,5,8,3]
         X.count(3)
```

```
Out[74]: 4
```

3. Tuples

are used to store multiple items in a single variable. A tuple is one of 4 built-in data types in Python used to store collections of data, the other 3 are List, Set, and Dictionary, all with different qualities and usage. A tuple is a collection that is ordered and unchangeable.

```
In [75]: X=(1,2,3,4)
         X
Out[75]: (1, 2, 3, 4)
```

We can see that the element in the second position is 3.

```
In [76]: X=(1,2,3,4)
         X[2]
Out[76]: 3
```

The number of elements in this tuple is

```
In [77]: X=(1,2,3,4)
         len(X)
Out[77]: 4
```

Maximum and the minimum elements in tuple are

```
In [78]: X=(1,3,6,2,3,9,7,3,5,8,3)
         max(X)
Out[78]: 9

In [79]: min(X)
Out[79]: 1
```

the “+” and “*” operators in tuple

```
In [80]: X=(1,2,3,4)
         Y=(5,6,7,8)
         X+Y
Out[80]: (1, 2, 3, 4, 5, 6, 7, 8)

In [82]: X*2
Out[82]: (1, 2, 3, 4, 1, 2, 3, 4)
```

4. Dictionaries

are Python's implementation of a data structure that is more generally known as an associative array. A dictionary consists of a collection of key-value pairs. Each key-value pair maps the key to its associated value.

Syntax

Name={<key1>:<value!> , <key2>:<value2>}

Example

```
In [83]: car={"brand":"ford","model":"ecosport","year":2003};car
Out[83]: {'brand': 'ford', 'model': 'ecosport', 'year': 2003}
```

Properties

- Dictionaries are unordered. A dictionary contains key-value pairs but does not possess an order for the pairs.
- Keys are unique. Dictionary keys must be unique.
- Dictionaries in Python are **mutable/changeable**.
- Dictionary comprehension.

Dictionary operations

a. Key's method

You can obtain all the keys in the dictionary with the *key's method*.

```
In [2]: car.keys()
Out[2]: dict_keys(['brand', 'model', 'year'])
```

b. Value method

To obtain all the values in the dictionary.

```
In [3]: car.values()
Out[3]: dict_values(['ford', 'ecosport', 2003])
```

c. Items method

To obtain the keys and the associated values

```
In [4]: car.items()
Out[4]: dict_items([('brand', 'ford'), ('model', 'ecosport'), ('year', 2003)])
```

d. del

To remove an entry from the dictionary

```
In [33]: del car["brand"]
          car.items()
Out[33]: dict_items([('model', 'ecosport'), ('year', 2003)])
```


e. In

To test the presence of a key in the dictionary.

```
In [34]: "year" in car
```

```
Out[34]: True
```

```
In [35]: "brand" in car
```

```
Out[35]: False
```

f. get

To get the value associated with a key. It returns nothing if the dictionary contains no key.

```
In [36]: car.get("year")
```

```
Out[36]: 2003
```

g. set default

It returns the value of the item with the specified key. If the key does not exist, insert the key, with the specified value, see the example below.

```
In [44]: car.setdefault("color","red")
```

```
Out[44]: 'red'
```

```
In [47]: car.items()
```

```
Out[47]: dict_items([('model', 'ecosport'), ('year', 2003), ('color', 'red')])
```

h. copy

To copy a dictionary. In this example, we can copy the dict: car to my_new_car.

```
In [48]: my_new_car=car.copy()  
my_new_car
```

```
Out[48]: {'model': 'ecosport', 'year': 2003, 'color': 'red'}
```

i. len

It returns the number of items in the dictionary.

```
In [49]: len(car)
```

```
Out[49]: 3
```

5. Set

Sets are used to store multiple items in a single variable. A set is a collection which is *unordered*, *changeable**, and *unindexed*.

* Note: Set *items* are unchangeable, but you can remove items and add new items.

```
In [64]: flower={"rose","lotus",22,"jasmine"}
         flower
```

```
Out[64]: {22, 'jasmine', 'lotus', 'rose'}
```

Set operations

a. add

To add a new element to the set.

```
In [66]: flower.add("lilly")
         flower
```

```
Out[66]: {22, 'jasmine', 'lilly', 'lotus', 'rose'}
```

b. remove

To remove an item from the set.

```
In [75]: flower.remove("lotus")
         flower
```

```
Out[75]: {22, 'jasmine', 'lilly', 'rose'}
```

c. In- (membership operator)

To check the presence of an element.

```
In [76]: "lilly" in flower
```

```
Out[76]: True
```

d. Union

To find the union of two sets.

```
In [77]: x={1,2,3,6,5}
         y={3,8,9,2}
         x|y
```

```
Out[77]: {1, 2, 3, 5, 6, 8, 9}
```

e. Intersection-common-&

To find the intersection of two sets.

```
In [78]: x={1,2,3,6,5}
         y={3,8,9,2}
         x&y
```

```
Out[78]: {2, 3}
```

DATA TYPE CONVERSIONS

1. List to tuple

```
In [168]: list1=["oo","pp"]
```

```
In [169]: tuple1=(*list1,)
          tuple1
```

```
Out[169]: ('oo', 'pp')
```

It can also be used

```
In [167]: tuple2=tuple(list1)
```

2. Tuple to list

```
In [171]: tuple1=(1,2,3)
          list1=[*tuple1]
          list1
```

```
Out[171]: [1, 2, 3]
```

It can also be used

```
In [172]: list2=list(tuple1)
```

3. Dictionary to list

```
In [173]: car={"brand":"ford","model":"ecosport","year":2003};car
```

```
Out[173]: {'brand': 'ford', 'model': 'ecosport', 'year': 2003}
```

```
In [175]: list1=[*car.items()]
```

```
In [176]: list1
```

```
Out[176]: [('brand', 'ford'), ('model', 'ecosport'), ('year', 2003)]
```

4. List to dictionary

```
In [179]: list1=["name","age","class"]
          list2=["anu","15","10.b"]
          dict1=dict(zip(list1,list2))
```

```
In [180]: dict1
```

```
Out[180]: {'name': 'anu', 'age': '15', 'class': '10.b'}
```

5. Set to list

```
In [191]: set1={"hello","good","morning"};set1
```

```
Out[191]: {'good', 'helllo', 'morning'}
```

```
In [192]: list1=[*set1];list1
```

```
Out[192]: ['good', 'morning', 'helllo']
```

6. List to set

```
In [193]: list1=["how","are","you"];list1
```

```
Out[193]: ['how', 'are', 'you']
```

```
In [194]: set1=set(list1);set1
```

```
Out[194]: {'are', 'how', 'you'}
```

LISTS VS. TUPLES VS. SETS VS. DICTIONARIES

	Lists	Tuples	Sets	Dictionaries
Ordering	Ordered	Ordered	Unordered	Ordered Unordered before Python 3.7
Indexing	Indexed	Indexed	Not Indexed	Keyed
Mutability	Mutable	Immutable	Mutable Only Adding and Removing	Mutable
Duplicates Allowed	Yes	Yes	No	Yes Only in values and not in keys
Types Allowed	Mutable and Immutable	Mutable and Immutable	Only Immutable	Only Immutable In keys